A time instance can be converted to a time instance in a different time zone with getutc or getlocal, both of which have corresponding methods utc and localtime, which change the receiver in place. You can convert to a Date or DateTime with to_date or to_datetime.

You can convert a Time instance to an Array using to_a, which returns a 10-element array made up of the attributes [sec, min, hour, day, month, year, wday, yday, dst?, zone]. This array can be passed back to some of the Time constructors to create a new Time instance. Although you cannot convert a Time to a hash directly, Time does implement deconstruct_keys so you can use a Time instance in pattern matching. For example:

```
x = Time.now
case x
in {month: 3|4|5, day:}
  puts "Spring is here and it's the #{day}th"
else
  puts "It's not winter"
end
```

produces:

```
Spring is here and it's the 18th
```

You can convert a Time instance to a string in several ways. The most general is strftime, which takes a format string as an argument. The format string uses the characters shown in Table 14, directives, on page 498 to insert parts of the actual time in the string. Following are specific string formatting methods:

- asctime (also ctime) formats as %a %b %e %T %Y, which has a shortcut %c.
- httpdate formats as %a, %d %b %Y %T GMT.
- inspect formats as %Y-%m-%d %H:%M:%S %N %z.
- rfc2822 formats as %a, %-d %b %Y %T %z.
- to_s formats as %Y-%m-%d %H:%M:%S %z, which is the same as inspect except for the subseconds.

For example:

```
require "time"
t = Time.now
t.strftime("%m/%d/%y")                # => "03/02/23"
t.strftime("%a %b %e %H:%M:%S %Z %Y") # => "Thu Mar  2 09:07:03 CST 2023"
t.ctime                               # => "Thu Mar  2 09:07:03 2023"
t.httpdate                            # => "Thu, 02 Mar 2023 15:07:03 GMT"
t.inspect                             # => "2023-03-02 09:07:03.883396 -0600"
t.rfc2822                             # => "Thu, 02 Mar 2023 09:07:03 -0600"
t.to_s                                # => "2023-03-02 09:07:03 -0600"
```

Creating Date and DateTime Instances

The date library implements classes Date and DateTime, which provide a comprehensive set of facilities for storing, manipulating, and converting dates with or without time components. The classes can represent and manipulate civil, ordinal, commercial, Julian, and standard dates, starting January 1, 4713 BCE. The DateTime class extends Date with hours, minutes, seconds, and fractional seconds, and it provides some support for time zones. The classes also provide support for parsing and formatting date and datetime strings. To use them, you need to require "date".